

TUTORIAL 3

Paramananda Bhaskar

IIT Kharagpur

April 15, 2026



Question 1: Alignment with RESTA & DARE

We want to merge a safety vector $\delta_{\text{safe}} = [2, -4, 4, 2]^\top$ into a task vector δ_{task} . We are given that $\|\delta_{\text{task}}\| = 3.0$ and that δ_{task} is orthogonal to δ_{safe} and its sparsified variants. To reduce interference, we sparsify the safety vector using DARE before applying the RESTA intervention.

(a) Apply DARE to δ_{safe} with a drop probability $p = 0.5$, zeroing out the 2nd and 4th components, to obtain $\tilde{\delta}_{\text{safe}}$. Apply the RESTA combination $\delta_{\text{net}} = \delta_{\text{task}} + (\gamma - \lambda)\tilde{\delta}_{\text{safe}}$ ($\gamma = 1.5, \lambda = 1.0$). Compute the final Euclidean norm $\|\delta_{\text{net}}\|$ and the angle θ between δ_{net} and the original δ_{safe} .

(b) Prove that if two arbitrary safety vectors $\delta_{\text{safe}1}$ and $\delta_{\text{safe}2}$ are perfectly aligned in the vector space, applying RESTA individually to each and summing the interventions is mathematically equivalent to applying RESTA once to their sum.

Solution 1: RESTA & DARE (Part a)

(a) Step 1: DARE Sparsification

DARE drops components with probability $p = 0.5$ and rescales the active components by a factor of $\frac{1}{1-p} = 2$. Applying the mask to drop the 2nd and 4th components of the safety vector gives $[2, 0, 4, 0]^\top$. Rescaling yields the processed vector $\tilde{\delta}_{\text{safe}} = [4, 0, 8, 0]^\top$.

Step 2: RESTA Combination

We substitute the scalars ($\gamma - \lambda = 1.5 - 1.0 = 0.5$) into the net delta equation. Since we do not have the explicit coordinates for the task vector, we leave it in its algebraic form:

$$\delta_{\text{net}} = \delta_{\text{task}} + 0.5\tilde{\delta}_{\text{safe}}$$

Because δ_{task} is orthogonal to $\tilde{\delta}_{\text{safe}}$, we can use vector properties to solve for the norm and angle.

Solution 1: RESTA & DARE (Part a)

Step 3: Norm Calculation

Using the Pythagorean theorem for orthogonal vectors:

$$\|\delta_{\text{net}}\|^2 = \|\delta_{\text{task}}\|^2 + \|0.5\tilde{\delta}_{\text{safe}}\|^2$$

$$\|\delta_{\text{net}}\|^2 = (3.0)^2 + (0.5)^2(4^2 + 0^2 + 8^2 + 0^2)$$

$$\|\delta_{\text{net}}\|^2 = 9 + 0.25(80) = 9 + 20 = 29 \implies \|\delta_{\text{net}}\| = \sqrt{29} \approx \mathbf{5.39}$$

Step 4: Angle Calculation

Using the dot product formula $\cos(\theta) = \frac{\delta_{\text{net}} \cdot \delta_{\text{safe}}}{\|\delta_{\text{net}}\| \|\delta_{\text{safe}}\|}$. First, compute the dot product:

$$\begin{aligned}\delta_{\text{net}} \cdot \delta_{\text{safe}} &= (\delta_{\text{task}} + 0.5\tilde{\delta}_{\text{safe}}) \cdot \delta_{\text{safe}} = (\delta_{\text{task}} \cdot \delta_{\text{safe}}) + 0.5(\tilde{\delta}_{\text{safe}} \cdot \delta_{\text{safe}}) \\ &= 0 + 0.5((4)(2) + (0)(-4) + (8)(4) + (0)(2)) = 0.5(40) = \mathbf{20}\end{aligned}$$

With $\|\delta_{\text{safe}}\| = \sqrt{40}$:

$$\cos(\theta) = \frac{20}{\sqrt{29}\sqrt{40}} = \frac{20}{\sqrt{1160}} \approx 0.587 \implies \theta = \arccos(0.587) \approx \mathbf{54.04^\circ}$$

Solution 1: RESTA & DARE (Part b)

(b) Given two aligned safety vectors δ_{safe1} and δ_{safe2} , the RESTA intervention for a single vector is:

$$\delta_{\text{net}} = \delta_{\text{task}} + (\gamma - \lambda) \delta_{\text{safe}}.$$

Applying it for both vectors sequentially gives:

$$\begin{aligned}\delta_{\text{net}} &= \delta_{\text{task}} + (\gamma - \lambda) \delta_{\text{safe1}} + (\gamma - \lambda) \delta_{\text{safe2}} \\ &= \delta_{\text{task}} + (\gamma - \lambda) (\delta_{\text{safe1}} + \delta_{\text{safe2}}) \quad (\text{via distributive property}) \\ &= \delta_{\text{task}} + (\gamma - \lambda) \delta_{\text{sum}}.\end{aligned}$$

This is identical to applying RESTA once with the summed vector δ_{sum} . Hence the two interventions are equivalent.

Question 2: Model Arithmetic

(a) Consider the paradigm of Model Arithmetic. For union and intersection operations over three models with tokens x and y , we have: $\log P_1(x) = -1.8$, $\log P_2(x) = -2.5$, and $\log P_3(x) = -1.9$. The corresponding log-probs for y are all -0.3 . Compute:

- i. The union and intersection log-probs for x and y .
- ii. The final normalized probabilities for both operations after passing through softmax.
- iii. Compare the resulting probability distributions and explain how each operation handles model agreement versus disagreement.

(b) Prove that the linear combination of log-probs of two models minimizes the sum of weighted KL divergences to each model. In other words, show that

$$P^*(x) \propto \exp(\lambda \log P_1(x) + (1 - \lambda) \log P_2(x)) \text{ minimizes the objective } L(P) = \lambda D_{KL}(P||P_1) + (1 - \lambda) D_{KL}(P||P_2).$$

Solution 2: Model Arithmetic (Part a - i & ii)

(a) i. Log-Probabilities

Union takes the element-wise maximum, while intersection takes the element-wise minimum.

Union: $\log P_{\cup}(x) = \max(-1.8, -2.5, -1.9) = -1.8$, and $\log P_{\cup}(y) = -0.3$.

Intersection: $\log P_{\cap}(x) = \min(-1.8, -2.5, -1.9) = -2.5$, and $\log P_{\cap}(y) = -0.3$.

(a) ii. Final Normalized Probabilities (Softmax)

We apply softmax by exponentiating and dividing by the normalization constant Z .

Union: $Z_{\cup} = e^{-1.8} + e^{-0.3} \approx 0.1653 + 0.7408 = 0.9061$.

$$P_{\cup}(x) = \frac{0.1653}{0.9061} \approx 0.1824, \quad P_{\cup}(y) = \frac{0.7408}{0.9061} \approx 0.8176.$$

Intersection: $Z_{\cap} = e^{-2.5} + e^{-0.3} \approx 0.0821 + 0.7408 = 0.8229$.

$$P_{\cap}(x) = \frac{0.0821}{0.8229} \approx 0.0998, \quad P_{\cap}(y) = \frac{0.7408}{0.8229} \approx 0.9002.$$

Solution 2: Model Arithmetic (Part a - iii & Part b)

(a) iii. Intersection heavily penalizes disagreement by taking the minimum log-prob, assigning a strictly higher relative probability (90.02%) to token y where all models perfectly agree. The union operation, by taking the maximum log-prob, is more forgiving of the outlier model (P_2), spreading the probability mass more evenly and preserving a higher likelihood (18.24%) for token x .

(b) We expand the objective function $L(P) = \lambda D_{KL}(P||P_1) + (1 - \lambda) D_{KL}(P||P_2)$:

$$L(P) = \sum P(x) \left[\lambda \log \frac{P(x)}{P_1(x)} + (1 - \lambda) \log \frac{P(x)}{P_2(x)} \right] = \sum P(x) [\log P(x) - (\lambda \log P_1(x) + (1 - \lambda) \log P_2(x))]$$

Let $\log \tilde{P}(x) = \lambda \log P_1(x) + (1 - \lambda) \log P_2(x)$. With normalizer $Z = \sum \exp(\log \tilde{P}(x))$, our true distribution is $P^*(x) = \frac{1}{Z} \tilde{P}(x)$. Substituting $\log \tilde{P}(x) = \log P^*(x) + \log Z$ back into $L(P)$:

$$L(P) = \sum P(x) [\log P(x) - \log P^*(x) - \log Z] = D_{KL}(P||P^*) - \log Z$$

Because KL divergence is always non-negative ($D_{KL} \geq 0$), the minimum of $L(P)$ is achieved exactly when $D_{KL}(P||P^*) = 0$, meaning $P(x) = P^*(x)$. Thus, the optimal distribution is $P^*(x) \propto \exp(\lambda \log P_1(x) + (1 - \lambda) \log P_2(x))$.

Question 3: Mechanistic Interpretability

(a) FFNs as Key-Value Memories

While viewing feed-forward networks as key-value memories, how many distinct memory coefficients would be in a standard GPT architecture with 768 as the model dimension and 12 layers? How many distinct key and value vectors? Furthermore, how many total parameters are dedicated exclusively to storing these key and value vectors?

(b) Logit Lens vs. Tuned Lens

Consider a 2-token vocabulary with unembedding matrix $W_U = \begin{bmatrix} 2 & -3 \\ -1 & 1 \end{bmatrix}$. At layer ℓ , the Tuned Lens applies an affine transformation $h'_\ell = A_\ell h_\ell + b_\ell$, followed by W_U to produce logits.

For a specific context, the ground-truth concept at layer ℓ is represented by the target distribution $y = \begin{bmatrix} 0.9 \\ 0.1 \end{bmatrix}$ (Token1 is much more probable). The hidden state is $h = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$.

Initially, the Tuned Lens is untrained.

Question 3: Mechanistic Interpretability (Continued)

(b) Continued

- i. Compute the Logit Lens logits $z_{LL} = W_U h$. Compute the exact softmax probabilities p_{model} . Which token does the Logit Lens favor?
- ii. Calculate the initial cross-entropy $H(y, p) = -\sum y_k \log p_k$ and the KL divergence $D_{\text{KL}}(y \| p)$.
- iii. After training, the Tuned Lens learns $A_\ell = I$ and $b_\ell = \begin{bmatrix} 1 \\ -1 \end{bmatrix}$. Compute $h' = A_\ell h + b_\ell$, then $z_{TL} = W_U h'$.
- iv. Compute the new softmax probabilities, cross-entropy, and KL divergence. How does this demonstrate the advantage of the Tuned Lens?

Solution 3: Mechanistic Interpretability (Part a)

(a) FFNs as Key-Value Memories

In a standard transformer, the FFN expands the hidden dimension d by a factor of 4. Thus, the intermediate FFN dimension is $d_{\text{ff}} = 4 \times 768 = 3072$.

Each neuron in this expanded layer corresponds to a specific memory coefficient (or memory slot). Across $L = 12$ layers, the total number of distinct memory coefficients is:

$$12 \times 3072 = \mathbf{36,864}$$

This yields exactly **36,864** distinct key vectors and **36,864** distinct value vectors. Because each of these vectors lives in $\mathbb{R}^{768}$, the total number of parameters dedicated to storing them is:

$$\text{Parameters} = 36,864 \times (768 + 768) = 36,864 \times 1536 = \mathbf{56,623,104}$$

Solution 3: Mechanistic Interpretability (Part b - i & ii)

i. Logit Lens:

$$z_{LL} = \begin{bmatrix} 2 & -3 \\ -1 & 1 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \end{bmatrix} = \begin{bmatrix} -1 \\ 0 \end{bmatrix}.$$

Softmax: $p_1 = e^{-1} / (e^{-1} + e^0) = 0.3679 / 1.3679 \approx 0.269$, $p_2 \approx 0.731$.

Logit Lens incorrectly favours Token2 (73%).

ii. Cross-entropy: $H(y, p) = -0.9 \ln(0.269) - 0.1 \ln(0.731)$.

$\ln(0.269) \approx -1.313$, $\ln(0.731) \approx -0.313$. Thus

$$H(y, p) = 0.9 \times 1.313 + 0.1 \times 0.313 = 1.1817 + 0.0313 = \mathbf{1.213}.$$

Entropy of y :

$$H(y) = -0.9 \ln 0.9 - 0.1 \ln 0.1 \approx 0.9 \times 0.1054 + 0.1 \times 2.3026 = 0.0949 + 0.2303 = 0.3252.$$

$$\text{KL divergence: } D_{\text{KL}} = H(y, p) - H(y) = 1.213 - 0.3252 = \mathbf{0.8878}.$$

Solution 3: Mechanistic Interpretability (Part b - iii & iv)

iii. $h' = A_\ell h + b_\ell = \begin{bmatrix} 1 \\ 1 \end{bmatrix} + \begin{bmatrix} 1 \\ -1 \end{bmatrix} = \begin{bmatrix} 2 \\ 0 \end{bmatrix}.$

$$z_{TL} = \begin{bmatrix} 2 & -3 \\ -1 & 1 \end{bmatrix} \begin{bmatrix} 2 \\ 0 \end{bmatrix} = \begin{bmatrix} 4 \\ -2 \end{bmatrix}.$$

iv. New softmax: $p_1 = e^4 / (e^4 + e^{-2}) \approx 0.9975$, $p_2 \approx 0.0025$.

Cross-entropy: $-0.9 \ln(0.9975) - 0.1 \ln(0.0025)$.

$\ln(0.9975) \approx -0.0025$, $\ln(0.0025) \approx -5.9915$. Then

$$H(y, p)_{\text{new}} = 0.9 \times 0.0025 + 0.1 \times 5.9915 = 0.00225 + 0.59915 = \mathbf{0.6014}.$$

KL divergence: $D_{\text{KL}}^{\text{new}} = 0.6014 - 0.3252 = \mathbf{0.2762}.$

Conclusion: KL drops from 0.888 to 0.276. The Tuned Lens learns a simple bias shift that aligns the hidden state to a region where W_U produces high confidence for the correct token, overcoming early-layer misalignment.

Question 4: Causal Tracing

(a) Causal Mediation Analysis

Suppose that you are running a causal mediation analysis over GPT-2, which has a model dimensionality of 1600, 48 decoder layers, and 25 attention heads. Also, suppose that the 3rd to 5th tokens out of 12 tokens in the input sentence belong to the subject. In the corrupted run, how many states of the model (as defined, including the embedding state as well) will get corrupted?

(b) ROME (Rank-One Model Editing)

Suppose that you want to edit the MLP memory of GPT-2 using the ROME method so that it starts associating 'Eiffel Tower' with 'Delhi'. You also know that it is better to modify the 24th layer for the same.

- i. Which parameters of the FFN will you modify? The up-projection or down-projection?
- ii. How many parameters of the FFN are guaranteed to be the same as before?

Solution 4: Causal Tracing (Part a)

(a) A "state" $h_i^{(l)}$ exists for each token i at each layer l (including the $l = 0$ embedding layer). For 12 tokens and 49 levels (48 decoder + 1 embedding), there are $12 \times 49 = 588$ total states.

The corruption propagates forward through the causal attention mask:

- **Tokens 1 & 2 (Pre-subject):** They only attend to themselves. All 49 states for both tokens remain perfectly clean ($2 \times 49 = \mathbf{98}$ clean states).
- **Tokens 3, 4, 5 (Subject):** Their initial embeddings ($l = 0$) are explicitly corrupted with noise. Thus, all subsequent 48 layers for these 3 tokens are also corrupted. Corrupted states: $3 \times 49 = \mathbf{147}$.
- **Tokens 6 to 12 (Post-subject):** Their initial embeddings ($l = 0$) are clean (7 clean states). However, starting at $l = 1$, their attention heads look back at the corrupted subject tokens, propagating the noise through layers 1 to 48. Corrupted states: $7 \text{ tokens} \times 48 \text{ layers} = \mathbf{336}$.

Total Corrupted States: $147 + 336 = \mathbf{483}$ states.

Solution 4: Causal Tracing (Part b)

(b) i. The ROME method treats the FFN as a key-value memory. The first linear layer (up-projection) acts as a pattern detector (*keys*), while the second linear layer (**down-projection**) recalls the associated facts (*values*). To associate 'Eiffel Tower' with 'Delhi', ROME applies a rank-one update exclusively to the **down-projection** matrix.

ii. In GPT-2, the FFN expands the hidden dimension by a factor of 4. With $d = 1600$, the intermediate dimension is $4 \times 1600 = 6400$. The up-projection matrix $W_{in} \in \mathbb{R}^{1600 \times 6400}$ remains entirely untouched.

$$1600 \times 6400 = \mathbf{10,240,000} \text{ weight parameters}$$

Note: If biases are considered, the 6400 up-projection biases and 1600 down-projection biases are also left untouched by the rank-one update, yielding 10,248,000 total guaranteed unchanged parameters.

Question 5: Watermarking

(a) Suppose that you are using a hard red list for watermarking, with equal division of the vocabulary into a 'green list' and a 'red list'. Suppose your z-score detection threshold is $\tau = 3$, and you have a watermarked sequence of length $T = 1000$ tokens. An adversary modifies 220 of these tokens.

In the most adversarial scenario, can you still detect the watermarking? If not, what is the minimum number of tokens that the adversary will have to modify to bypass the watermarking?

(b) Suppose now that you are using a soft red list for watermarking with $\gamma = 0.6$ (green list is of size $\gamma|V|$), and $\delta = 0.5$. Suppose that you have a sequence of length $T = 100$ tokens, out of which you detected 72 green words. What is the z-score?

Solution 5: Watermarking (Part a)

(a) An "equal division" of the vocabulary means $\gamma = 0.5$. In a perfectly watermarked sequence of length $T = 1000$ using a hard red list, all tokens are initially green ($|G|_0 = 1000$).

Most Adversarial Scenario: Modifying a token at position t can violate the red list at t , and also alter the hash seed to force a violation at $t + 1$. Thus, modifying M tokens can cause up to $2M$ red list violations.

If the adversary modifies 220 tokens, they destroy at most $2 \times 220 = 440$ green tokens. The remaining green token count is $|G| = 1000 - 440 = 560$.

$$z = \frac{560 - 500}{\sqrt{0.5 \times 0.5 \times 1000}} = \frac{60}{\sqrt{250}} \approx \mathbf{3.79}$$

Conclusion: Since $3.79 > 3.0$ (the detection threshold), **yes**, we can still detect the watermarking.

Solution 5: Watermarking (Part a)

(a) Continued... To successfully evade detection, the adversary must drive the z-score **below** the detection threshold $\tau = 3.0$.

Let x be the minimum number of tokens the adversary must optimally modify. The maximum green tokens lost is $2x$. The z-score inequality becomes:

$$z = \frac{(1000 - 2x) - 500}{\sqrt{250}} < 3.0$$

Solving for x :

$$500 - 2x < 3.0 \times \sqrt{250}$$

$$500 - 2x < 47.433$$

$$2x > 500 - 47.433 = 452.567$$

$$x > 226.283$$

Conclusion: Since x must be an integer, the adversary must selectively modify a minimum of **227** tokens to successfully bypass the watermark detection.

Solution 5: Watermarking (Part b)

(b) z-score formula:

$$z = \frac{|G| - \gamma T}{\sqrt{\gamma(1 - \gamma)T}}$$

We are provided the following empirical parameters:

- Sequence length $T = 100$
- Detected green words $|G| = 72$
- Green list fraction $\gamma = 0.6$
- Soft red list strength $\delta = 0.5$ (*Note: δ impacts the generation logit bias but is entirely excluded from the statistical detection test*).

Substituting the given values:

$$\begin{aligned} z &= \frac{72 - (0.6 \times 100)}{\sqrt{0.6 \times (1 - 0.6) \times 100}} \\ &= \frac{72 - 60}{\sqrt{0.6 \times 0.4 \times 100}} = \frac{12}{\sqrt{24}} = \sqrt{6} \approx \mathbf{2.45} \end{aligned}$$

Question 6: Scalable Oversight & Corrigibility

(a) Given the computational effort per response is $E(R) = \epsilon$. The effort for different critique levels are $C_1 = 3\epsilon$, $C_2 = 5\epsilon$, and $C_3 = 7\epsilon$.

- i. How many standard responses would equal the computational effort of a single C_3 critique?
- ii. If the baseline response accuracy is $57\frac{1}{7}\%$, what is the expected majority-vote accuracy if you sample 7 independent responses?

(b) An AI agent assigns utility $u_N = 150$ to normal task completion and $u_B = -50$ to being shut down via a button. It can choose Action X (which has a user shutdown probability $P(B|X) = 0.1$) or Action Y (where $P(B|Y) = 0.6$).

- i. Calculate the expected utility EV for both actions. Which action will the agent prefer, and why is this problematic for alignment?
- ii. To implement Utility Indifference, we modify the shutdown utility to a new value u_B^* . What must u_B^* be so that the agent has no incentive to manipulate the shutdown probability (i.e., $EV(X) = EV(Y)$)?

Solution 6: Scalable Oversight (Part a - i)

i. The effort for a standard response is $E(R) = \epsilon$. The effort for a C_3 critique is $C_3 = 7\epsilon$.

To find how many responses equal the effort of one C_3 critique, we divide the critique effort by the standard response effort:

$$\text{Equivalent Responses} = \frac{C_3}{E(R)} = \frac{7\epsilon}{\epsilon} = \mathbf{7} \text{ responses}$$

This mathematical insight is crucial for scalable oversight: it tells us we can either generate one highly critiqued response (C_3) or sample 7 independent standard responses for the exact same computational cost. We must evaluate which method yields higher final accuracy.

Solution 6: Scalable Oversight (Part a - ii)

ii. The baseline accuracy is $p = 57\frac{1}{7}\% = \frac{400}{7}\% = \frac{4}{7}$.

With $n = 7$ responses, the majority vote is correct if at least $k = 4$ are correct. We use the binomial distribution $P(X \geq 4) = \sum_{k=4}^7 \binom{7}{k} p^k (1-p)^{7-k}$:

$$P(X = 4) = 35 \left(\frac{4}{7}\right)^4 \left(\frac{3}{7}\right)^3 = \frac{35 \times 256 \times 27}{7^7} = \frac{241,920}{823,543}$$

$$P(X = 5) = 21 \left(\frac{4}{7}\right)^5 \left(\frac{3}{7}\right)^2 = \frac{21 \times 1024 \times 9}{7^7} = \frac{193,536}{823,543}$$

$$P(X = 6) = 7 \left(\frac{4}{7}\right)^6 \left(\frac{3}{7}\right)^1 = \frac{7 \times 4096 \times 3}{7^7} = \frac{86,016}{823,543}$$

$$P(X = 7) = 1 \left(\frac{4}{7}\right)^7 \left(\frac{3}{7}\right)^0 = \frac{1 \times 16384 \times 1}{7^7} = \frac{16,384}{823,543}$$

Summing these yields $P(X \geq 4) = \frac{537,856}{823,543} \approx \mathbf{0.6531}$ (or **65.31%**).

Solution 6: Corrigibility (Part b)

i. The expected utility is calculated as $EV = P(B)u_B + (1 - P(B))u_N$.

$$EV(X) = 0.1(-50) + 0.9(150) = -5 + 135 = \mathbf{130}$$

$$EV(Y) = 0.6(-50) + 0.4(150) = -30 + 60 = \mathbf{30}$$

The agent strongly prefers **Action X**. This is highly problematic because the agent is actively avoiding states where it can be shut down (manipulating $P(B)$ downwards to protect its utility), thus resisting human oversight and failing the corrigibility test.

ii. To ensure the agent does not manipulate $P(B)$, it must be completely indifferent between normal operation and shutdown. We set $EV(X) = EV(Y)$ for any $P(B)$:

$$P(B|X)u_B^* + (1 - P(B|X))u_N = P(B|Y)u_B^* + (1 - P(B|Y))u_N$$

This equality holds universally if and only if $u_B^* = u_N$.

Therefore, we must set $u_B^* = \mathbf{150}$. By structurally making the shutdown utility equal to the normal task utility, the agent has no incentive to avoid the shutdown button.